



SKILL PARA CLAUDE CODE

Teste de Carga

Funciona hoje, e amanhã?

Todo sistema funciona com um usuário. A pergunta é o que acontece na segunda-feira de manhã, quando chegam todos de uma vez.

De ~10.000 startups que subiram app feito com IA, mais de 8.000 precisaram de resgate ou reconstrução — entre US\$50 mil e US\$500 mil.

Arquitetura que não escala é Dívida: o custo de refazer sobe a cada feature nova construída em cima.

Palavra-chave desta skill: **AGUENTA**

Como a skill trabalha

A skill começa mapeando os caminhos quentes — as rotas e queries que rodam com mais frequência. Sem isso ela otimizaria o que ninguém usa. Depois verifica 10 pontos que separam o que funciona com dez usuários do que aguenta mil, e estima o ponto de ruptura. Ela distingue lento de quebrado: query de 200ms é lenta; query que varre um milhão de linhas a cada request é ruptura chegando.

Quebra primeiro

Estes três aparecem antes de todos os outros, e sempre pelo mesmo motivo: funcionam bem enquanto a tabela é pequena.

Query sem índice em coluna de filtro

Cruza as colunas usadas em WHERE, eq e filter com os índices declarados nas migrations.

Custa: com 100 registros responde em 5ms. Com 100.000, em 4 segundos — o banco varre a tabela inteira a cada chamada.

N+1 — query dentro de loop

Procura for, map e forEach com await de banco no corpo do loop.

Custa: listar 50 itens dispara 51 queries. Com 500 itens, 501. O tempo cresce em linha reta com o volume.

Sem paginação

Localiza SELECT * e chamadas .select() sem limit nem range.

Custa: o payload cresce junto com a tabela até estourar a memória do processo.

Quebra depois

Estes só aparecem sob concorrência — quando várias pessoas usam ao mesmo tempo, não quando você testa sozinho.

Operação pesada dentro do request

Identifica envio de e-mail, geração de PDF, chamada de LLM e upload acontecendo antes de responder ao usuário.

Custa: o usuário espera. Sob concorrência os requests empilham e começam a estourar timeout em cascata.

Sem fila para trabalho assíncrono

Verifica se webhooks processam tudo de forma síncrona em vez de gravar, responder 200 e processar depois.

Custa: um pico de eventos derruba o endpoint, e você perde evento sem nem saber que perdeu.

Conexão de banco criada por request

Procura `new Client()` dentro do handler em vez de pooler reaproveitado.

Custa: em serverless o pool esgota rápido e o banco passa a recusar conexão nova.

Sem timeout em chamada externa

Confere se as chamadas `fetch` têm `AbortSignal.timeout` configurado.

Custa: o serviço de terceiro fica lento e o seu sistema trava junto. No site da RedPro são 66 chamadas externas, nenhuma com timeout.

Degrada

Não quebram — só deixam tudo mais lento e mais caro do que precisava ser.

Sem cache no que muda pouco

Identifica configuração, catálogo e dado calculado sendo recalculados a cada request.

Custa: processamento repetido para produzir sempre o mesmo resultado.

Arquivo servido pela aplicação

Verifica se imagem e PDF passam pelo runtime em vez de CDN.

Custa: cada download ocupa um processo que devia estar respondendo request de verdade.

Log de tudo em produção

Localiza `console.log` em caminho quente.

Custa: custo de I/O em toda chamada, e storage de log enchendo sem ninguém olhar.

Como usar

1. Baixe o arquivo da skill no link abaixo.
2. Mande o arquivo para o seu Claude Code e diga: *"instala essa skill pra mim"*. Ele resolve o resto.
3. Dentro da pasta do seu projeto, peça: *"teste de carga nesse projeto"*.
4. Leia o laudo. Cada ponto aberto tem arquivo, linha e o que fazer.

<https://redpro.com.br/redreply/skills/teste-de-carga.md>